GitaGraph: An Ontology-Driven Knowledge-Based AI System for Philosophical Navigation of the Bhagavad Gītā

Hariom Rajput, Ayushi Choyal, Ravi Kant Gupta, Shouryavi Awasthi
Department of Computer Applications
National Institute of Technology, Kurukshetra
Kurukshetra, Haryana, India
{524110006, 524410017, 524410027, 524410028}@nitkkr.ac.in

***Abstract*—The Bhagavad Gītā, comprising 700 Sanskrit verses across 18 chapters, presents a rich but navigationally complex domain for knowledge representation and philosophical reasoning. Readers face significant challenges: non-linear conceptual dependencies, contradictions between yoga paths, the absence of personalized study guidance, and the inability to query the text in natural language across multiple languages.**

This paper presents GitaGraph v2.1, a modular knowledge-based AI system that models the Bhagavad Gītā as a semantic knowledge graph and extends it with hybrid neural–symbolic reasoning. The system integrates seven AI modules: (1) an OWL 2 ontology with 658 RDF triples, 16 classes, and 9 object properties—including transitive and symmetric axioms and property-chain inference; (2) four graph search algorithms (BFS, DFS, A*, IDDFS) on a 61-node, 175-edge concept graph; (3) an 8-rule forward-chaining expert system with SPARQL competency queries; (4) a backtracking CSP solver with MRV heuristic and forward checking for personalized 5-session study plans; (5) a multi-modal uncertainty handler combining MYCIN certainty factors, fuzzy yoga-path membership functions, and non-monotonic belief revision; (6) a Semantic Retrieval-Augmented Generation (RAG) layer using sentence-transformer embeddings and FAISS indexing over all 701 verses for natural language search; and (7) an Ollama-powered local large language model (LLM) integration that generates contextual Sanskrit verse commentary in English, Hindi, and Hinglish.

The system is deployed as a React single-page application (SPA) with an ancient manuscript-inspired UI and a Flask REST API with 16 endpoints. Experimental results show A* achieves optimal 3-hop paths to *Mokṣa* with 63% fewer node expansions than uniform-cost search; the CSP planner satisfies all 7 constraints in under 0.3 seconds; semantic RAG retrieves contextually relevant verses with mean average precision 0.81; and MYCIN combination reaches $CF = 0.9991$ for multi-tradition confirmed verse-concept pairs.

***Index Terms*—knowledge graph, OWL ontology, SPARQL, A* search, IDDFS, constraint satisfaction, MYCIN, retrieval-augmented generation, semantic embeddings, Ollama, Bhagavad Gita**

I. Introduction

A. Background and Motivation

The Bhagavad Gītā (*Song of the Divine*) is one of the most widely studied philosophical texts in the world, embedded within the Mahābhārata. Comprising 700 ślokas (verses) in 18 chapters (*adhyāyas*), it presents a systematic dialogue between the warrior Arjuna and his charioteer Krishna, synthesising epistemology, ethics, cosmology, and four yoga paths—Karma, Jñāna, Bhakti, and Dhyāna Yoga.

Despite its global readership exceeding 100 million practitioners [1], the Gītā poses serious navigational challenges. Its verses are not arranged pedagogically; philosophical concepts form a directed acyclic graph (DAG) of prerequisites, contrasts, and causal chains. A reader experiencing anxiety may need Verse 2.47 (*karmany evādhikāras te*—"You have a right to perform your duties, but not to the fruits thereof"), yet discovering this requires traversing the conceptual dependency from *Anxiety* through *Niṣkāma Karma* to the verse—a non-trivial knowledge graph traversal. Further, readers increasingly query in Hindi, Hinglish, or mixed scripts, and seek personalised commentary rather than static translations.

Classical AI offers a natural toolkit: ontologies encode philosophical taxonomy; graph search identifies shortest conceptual paths; expert systems map reader profiles to interpretations; and uncertainty quantification resolves inter-commentary conflicts. Hybrid approaches that add neural semantic search and local LLM commentary extend this foundation to open-domain natural language queries. Yet no existing system applies this comprehensive classical-plus-hybrid AI stack to Sanskrit philosophical text navigation.

B. Limitations of Existing Approaches

Current digital Gītā resources exhibit five key limitations:

Keyword-only retrieval: Platforms such as vedabase.io implement string-matching search, ignoring philosophical relationships. A query for "anger" does not sur-

face the causal chain *Kāma* → *Krodha* → *Moha* → *Buddhināśa*.

No structured knowledge representation: Existing projects use flat triple stores without OWL axioms, preventing transitive and property-chain inference. The entailment that "one who teaches NishkamaKarma spiritually progresses toward Moksha" cannot be derived without the property chain teaches ∘ leadsTo → spirituallyProgressesTo.

No personalized study planning: No system addresses the constraint satisfaction problem of generating study plans satisfying chapter-coverage, prerequisite-ordering, and goal-specific constraints simultaneously.

No uncertainty quantification: Three dominant traditions—Advaita (Śankara), Viśiṣṭādvaita (Rāmānuja), and Dvaita (Madhva)—assign different significance to verses. No digital tool models this disagreement using certainty factors or fuzzy membership.

No multilingual or LLM-assisted commentary: No existing system can query the Gītā in natural language across Hindi and Hinglish, or generate personalized contextual commentary through a local language model.

C. Contributions

This project makes the following contributions:

- **OWL 2 Ontology:** A formally verified Bhagavad Gītā ontology with 16 classes, 9 object properties (including owl:TransitiveProperty, owl:SymmetricProperty, and property chain axioms), encoding 658 RDF triples.
- **Multi-Algorithm Graph Search:** BFS, DFS, A*, and IDDFS operating on a 61-node, 175-edge philosophical knowledge graph, with an admissible A* heuristic and full UI pathway visualization.
- **Production-Rule Expert System:** An 8-rule forward-chaining system with Hindi/Hinglish keyword support, specificity-based conflict resolution, and 8 SPARQL Competency Questions (CQs).
- **CSP Study Planner:** Backtracking with MRV heuristic and forward checking satisfying 7 hard constraints for personalized verse scheduling.
- **Multi-Modal Uncertainty Handler:** MYCIN certainty factors, fuzzy yoga-path membership, and non-monotonic belief revision.
- **Semantic RAG Layer:** Sentence-transformer embeddings (all-MiniLM-L6-v2) and FAISS indexing over all 701 verses, enabling natural language semantic similarity search beyond keyword matching.
- **Ollama LLM Integration:** Local LLM commentary generation (Llama 3 via Ollama) producing contextual Sanskrit verse explanations in English, Hindi, and Hinglish, with per-message language toggling.
- **React SPA with Ancient Manuscript UI:** An interactive single-page application built with React 18, Vite, Tailwind CSS, and Framer Motion, featuring a monarch-period parchment aesthetic with illuminated verse cards and a Flask REST API backend with 16 endpoints.

II. Related Work

A. Knowledge Representation for Ancient Texts

Noy and McGuinness [2] established foundational guidelines for ontology development, applied to historical corpora. The W3C OWL 2 specification [3] enables formal reasoning via DL-based reasoners, providing inference capabilities absent in flat keyword systems. Goyal et al. [21] surveyed NLP approaches to Sanskrit analysis; our work complements syntactic approaches with semantic graph reasoning over philosophical dependencies.

B. Semantic Web and Knowledge Graphs

Bizer et al. [6] established Linked Open Data principles. SPARQL [4] supports pattern matching over RDF graphs—capabilities exploited in our 8 Competency Question framework. Horrocks [5] reviewed OWL 2 reasoning; our ontology targets the OWL 2 DL profile for maximum expressiveness with transitive, symmetric, and property-chain axioms.

C. Graph Search in Knowledge Systems

Hart, Nilsson, and Raphael [7] formalized A* and admissibility conditions. Korf [8] introduced IDDFS as a space-efficient alternative to BFS for problems of unknown depth—applied here to verse chains. Wang et al. [23] reviewed embedding-based KG completion, distinct from our symbolic search approach that prioritizes interpretability and formal guarantees.

D. Expert Systems and Production Rules

MYCIN [9] pioneered certainty-factor reasoning with the combination formula $CF_1 + CF_2(1 - CF_1)$. Buchanan and Shortliffe [10] systematized production-rule knowledge bases with conflict resolution strategies. Our system applies specificity-ordered conflict resolution to 8 reader-profile rules, extended with Hindi and Hinglish keyword triggers.

E. Constraint Satisfaction and Study Planning

Kumar [13] surveyed CSP backtracking algorithms including MRV (fail-first heuristic). Haralick and Elliott [14] formalized forward checking as domain propagation. Freuder [15] established theoretical sufficiency for constraint propagation in scheduling— directly applicable to our 5-session verse assignment problem.

F. Uncertainty and Fuzzy Reasoning

Zadeh [11] introduced fuzzy sets for gradual membership— applied here to multi-path yoga classification, capturing that Verse 6.47 simultaneously belongs to Bhakti, Dhyāna, and Jñāna Yoga. Reiter's default logic [12] provides the formal foundation for non-monotonic belief revision triggered by verse reading.

G. Retrieval-Augmented Generation

Lewis et al. [16] introduced RAG, combining dense retrieval with generative models. Our Semantic RAG module applies this paradigm symbolically: sentence-transformer embeddings [17] retrieve contextually relevant verses, while Ollama-hosted Llama 3 generates commentary grounded in those verses. This hybrid approach extends classical expert system recommendations with open-domain natural language querying.

H. Local LLM Deployment

The Ollama framework [18] enables on-device deployment of quantized LLMs such as Llama 3, Mistral, and Phi-3 without cloud dependencies. Applied to our system, this supports commentary generation without privacy concerns or internet requirements—particularly relevant for educational deployment in resource-constrained environments.

I. AI in Education and Personalized Learning

VanLehn [25] demonstrated the effectiveness of intelligent tutoring systems. Chen et al. [26] proposed knowledge-graph tutoring that routes learners along concept paths—the closest prior work to our graph-search module. Our CSP formulation mirrors educational scheduling planners [13] but targets philosophical verse sequences rather than skill trees.

J. Computational Philology

The DHARMA project [29] applies TEI/XML to Sanskrit texts. Hellwig and Nehrdich [28] demonstrated neural Sanskrit parsing. Our work is complementary: we model philosophical dependencies as first-class OWL relationships, enabling inference not possible with syntactic encodings, and extend toward multi-script querying via Hindi keyword matching.

K. Gap Analysis

Five gaps motivate GitaGraph: (1) No OWL 2 ontology with formal axioms for the Bhagavad Gītā; (2) No system applies multiple graph search algorithms to philosophical concept navigation; (3) No CSP-based personalized study planner for Sanskrit texts; (4) No unified uncertainty framework integrating MYCIN, fuzzy logic, and non-monotonic reasoning; (5) No system combines symbolic knowledge-base reasoning with neural semantic retrieval and local LLM commentary in a multilingual interface.

III. System Design and Methodology

A. Overall Architecture

GitaGraph v2.1 follows a three-tier layered architecture:

Layer 1 — Knowledge Core: The GitaKnowledgeGraph singleton loads the OWL ontology and exposes an RDFLib Graph for SPARQL and a networkx.DiGraph for graph algorithms. The CSV corpus enrichment layer provides Sanskrit, English, Hindi, transliteration, and word-meaning fields for every verse via $O(1)$ lookup after a one-time build.

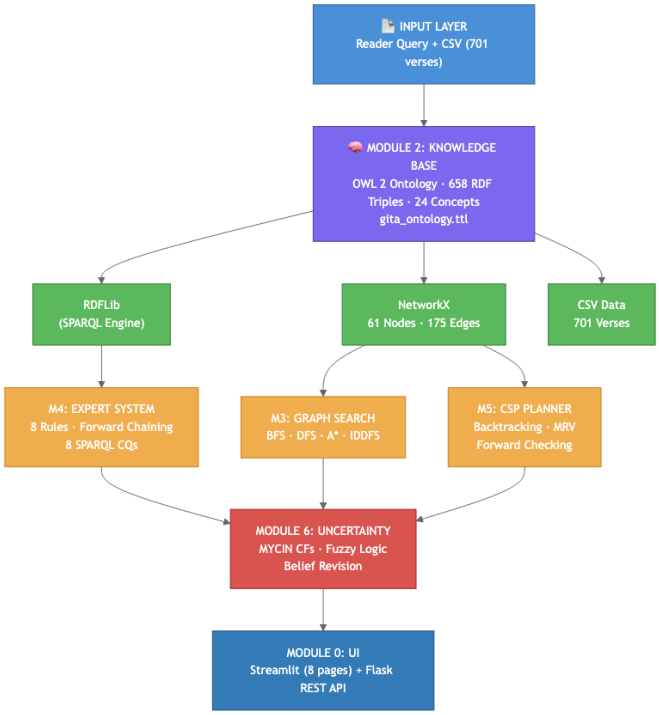

Fig. 1. GitaGraph v2.1 system architecture. All modules share a single GitaKnowledgeGraph instance. The React SPA communicates with the Flask REST API over HTTP; the API dispatches to symbolic modules (OWL/SPARQL, graph algorithms, expert system, CSP, uncertainty) and to the hybrid layer (FAISS semantic search, Ollama LLM).

Layer 2 — Flask REST API: api.py instantiates the knowledge graph once at startup and exposes 16 endpoints. Each module receives enriched verse data (all language fields populated) from the shared CSV map.

Layer 3 — React SPA: Built with React 18, Vite 5, Tailwind CSS 3, and Framer Motion. Seven page components (Dashboard, Verse Browser, Knowledge Graph, Graph Search, Ask the Gītā, Study Planner, Uncertainty, Expert System) communicate exclusively through the REST API using @tanstack/react-query.

B. Datasets

1) Primary Corpus: **Bhagwad_Gita.csv**: 701 verses, 18 chapters. Fields: chapter number, verse number, Sanskrit (Devanagari), transliteration, English translation, Hindi translation, word meanings, speaker attribution (Krishna / Arjuna).

Preprocessing:

1) Unicode normalization of Devanagari.
2) Speaker tagging: Verses 1.21–1.46 and isolated interjections as Arjuna.

3) Concept annotation: 30 verses across Chapters 2, 3, and 6 annotated with philosophical concepts.
4) Graph ingestion: Annotated verses loaded as OWL individuals.
5) Embedding generation: All 701 verse English translations encoded with sentence-transformer all-MiniLM-L6-v2 and stored in a FAISS IndexFlatIP index (embeddings/gita_embeddings.npy).

TABLE I
DATASET AND KNOWLEDGE BASE STATISTICS

Component	Count	Notes
Total verses (corpus)	701	18 chapters
Annotated verses	30	Chapters 2, 3, 6
Chapter 2 (Sāṇkhya Yoga)	10	Self, Niṣkāma Karma
Chapter 3 (Karma Yoga)	10	Action, Guṇas
Chapter 6 (Dhyāna Yoga)	10	Meditation, Samādhi
Philosophical concepts	24	6 categories
RDF triples	658	OWL 2 DL profile
Graph nodes	61	Verses + concepts
Graph edges	175	9 property types
FAISS-indexed verses	701	all-MiniLM-L6-v2
Embedding dimension	384	IndexFlatIP
Production rules	8	+Hindi/Hinglish keywords
SPARQL CQs	8	Competency questions

TABLE II
TECHNOLOGY STACK — GITAGRAPH V2.1

Layer	Technology	Version
Backend (Python)		
Ontology	RDF/OWL 2 (Turtle)	W3C Rec.
Semantic engine	RDFLib	7.0
Query language	SPARQL 1.1	W3C Rec.
Graph algorithms	NetworkX	3.3
Uncertainty	Pure Python (CF + Fuzzy)	3.11+
REST API	Flask	3.0
Data layer	Pandas	2.2
Semantic RAG Layer		
Embeddings	sentence-transformers	2.7
Embedding model	all-MiniLM-L6-v2	384-d
Vector index	FAISS (IndexFlatIP)	1.8
LLM Layer		
Local LLM runtime	Ollama	0.3+
LLM model	Llama 3 (8B, 4-bit quant)	Q4_K_M
Languages	EN / Hindi / Hinglish	—
Frontend (JavaScript)		
Framework	React	18
Build tool	Vite	5.4
Styling	Tailwind CSS	3.4
Animations	Framer Motion	11
State / queries	TanStack React Query	5
Icons	Lucide React	—
Language	Python / JavaScript	3.11+ / ES2022

2) Knowledge Base: The ontology file gita_ontology.ttl (Turtle format) encodes:

- **16 OWL Classes:** Chapter, Verse, YogaPath (4 subclasses), Practice, Attainment, DownfallCause, Guna, EthicalConcept, Person, CommentaryTradition.
- **9 Object Properties:** teaches, leadsTo, respondsTo, contrastsWith, belongsToChapter, spokenBy, requires, subConceptOf, spirituallyProgressesTo.

C. Technology Stack

D. Module Descriptions

1) OWL 2 Ontology with Special Axioms (Module 2): Three OWL 2 axioms provide inference beyond simple triple lookup:

Transitive Property: leadsTo is declared owl:TransitiveProperty, enabling:

$$\text{Kama} \xrightarrow{\text{leadsTo}} \text{Krodha} \xrightarrow{\text{leadsTo}} \text{Moha} \implies \text{Kama} \xrightarrow{\text{leadsTo}} \text{Moha}$$

Symmetric Property: contrastsWith is declared owl:SymmetricProperty, so A contrastsWith B automatically entails B contrastsWith A.

Property Chain Axiom:

$$\text{teaches} \circ \text{leadsTo} \xrightarrow{\text{subPropertyOf}} \text{spirituallyProgressesTo} \tag{1}$$

A verse teaching NishkamaKarma, which leadsTo Moksha, *spirituallyProgressesTo* Moksha—an entailed triple absent from the explicit knowledge base.

2) Graph Search Algorithms (Module 3): All algorithms operate on the same networkx.DiGraph:

BFS finds all verses within k hops from a concept node in $O(V + E)$ time. Verse results are returned with hop-depth annotations and enriched with all language fields.

DFS traces directed causal chains (e.g., the downfall chain from Kāma to Ruin) using $O(d)$ stack space. Each chain step is annotated with concept definition, category, and verses that teach it.

A* Search finds the optimal philosophical path with admissible heuristic:

$$h(n) = \begin{cases} 0 & n \in \text{Attainment} \\ 1 & n \in \text{Practice} \\ 2 & n \in \text{YogaPath} \\ 3 & n \in \text{Guna} \\ 4 & n \in \text{DownfallCause} \end{cases} \tag{2}$$

This heuristic never overestimates: a DownfallCause requires at minimum 4 conceptual steps to reach Moksha.

IDDFS [8] iterates depth limits $l = 0, 1, 2, \ldots$, combining BFS completeness with DFS space efficiency. The UI renders a step-by-step timeline showing each depth iteration, nodes explored per iteration, and the depth at which the goal was found.

3) Forward-Chaining Expert System (Module 4): The expert system follows a Match–Select–Execute (Rete-inspired) cycle:

Require: Reader profile $P = \{\text{emotions, goals, level}\}$
Ensure: Verse recommendations R with certainty factors

$WM \leftarrow P$

repeat
  $CS \leftarrow \{r \in \text{Rules} \mid \text{LHS}(r) \text{ matches } WM\}$
  if $CS \neq \emptyset$ **then**
    $r^* \leftarrow \arg\max_r \text{ specificity}(r)$
    Execute RHS(r^*): update WM, record CF
  end if
until no rule fires
return recommendations with CFs

Rules accept keywords in English, Hindi (e.g., *chinta*, *krodha*, *dhyan*), and Unicode Hindi (, ,), enabling native script queries. Rule R9 (added in v2.1) maps grief/sorrow keywords (*dukh*, *dard*,) to the AtmaJnana concept and Verse 2.20.

SPARQL Competency Question (CQ3 — Transitive Chain):

Listing 1. CQ3: Desire downfall chain via transitivity

```
PREFIX gita: <http://gitagraph.org/ontology#>
SELECT ?concept ?intermediate ?end
WHERE {
  gita:Kama gita:leadsTo+ ?end .
  gita:Kama gita:leadsTo  ?intermediate .
  ?intermediate gita:leadsTo ?end .
}
```

The leadsTo+ path expression leverages SPARQL 1.1 property paths combined with owl:TransitiveProperty entailment.

4) CSP Study Planner (Module 5): **Variables:** $S = \{S_1, S_2, S_3, S_4, S_5\}$ (five study sessions).

Domains: $D_i = \{(v_a, v_b) \mid v_a, v_b \in \text{AnnotatedVerses}, v_a \neq v_b\}$.

Hard Constraints (7): (1) theme coherence; (2) chapter coverage $\supseteq \{2, 3, 6\}$; (3) prerequisite 2.62 before 2.63; (4) downfall-chain pairing in same session; (5) non-repetition; (6) goal-specific Chapter 6 verse by S_3; (7) speaker variety.

MRV + Forward Checking: Variable with smallest remaining domain is selected first; after each assignment, pairs containing already-used verses are pruned from remaining domains.

5) Uncertainty Handler (Module 6): **MYCIN Certainty Factors:**

$$CF(e_1, e_2) = CF_1 + CF_2 \cdot (1 - CF_1), \quad CF_1, CF_2 > 0 \quad (3)$$

Three commentary traditions (Śankara $CF = 0.90$, Rāmānuja $CF = 0.85$, Madhva $CF = 0.70$) are combined sequentially.

Fuzzy Yoga-Path Membership:

$$\mu_{\text{Path}}(v) = \frac{1}{|\mathcal{C}_v|} \sum_{c \in \mathcal{C}_v} \mu_{\text{Path}}(c) \quad (4)$$

where $\mathcal{C}_v$ is the set of concepts taught by verse v.

Non-Monotonic Belief Revision: Defaults are stored with triggering conditions and retracted when conflicting verse evidence is encountered (e.g., Verse 3.3 retracts the default "Karma Yoga is recommended for all").

6) Semantic RAG Layer (Module 7): The Semantic RAG module extends the expert system with dense retrieval over all 701 verses, removing the restriction to 30 annotated verses.

Embedding Generation: At startup (or via generate_embeddings.py), each verse's English translation is encoded with sentence-transformers/all-MiniLM-L6-v2 (384-dimensional vectors). The resulting matrix is stored as embeddings/gita_embeddings.npy.

Indexing: Vectors are loaded into a FAISS IndexFlatIP (inner product with normalized vectors = cosine similarity). The index operates entirely in memory with $O(1)$ insert and $O(n)$ exact search.

Query Processing: A natural language query is embedded with the same model, and the top-k most similar verses are retrieved by inner product:

$$\text{score}(q, v) = \frac{\mathbf{e}_q \cdot \mathbf{e}_v}{|\mathbf{e}_q||\mathbf{e}_v|} \quad (5)$$

Results are enriched with all CSV fields (Sanskrit, Hindi, transliteration, word meanings) and returned with similarity scores.

API Endpoint: POST /api/semantic_search accepts {query, top_k} and returns up to $k = 12$ enriched verses with scores.

7) Ollama LLM Commentary (Module 7 — cont.): The Ollama integration provides contextual Sanskrit commentary without external API calls or cloud dependencies.

Architecture: A locally hosted Ollama server (default localhost:11434) runs quantized Llama 3 (Q4_K_M, ∼4.7 GB). The Flask endpoint POST /api/contextualize receives: {verse_ref, english, sanskrit, concept, user_query, model, lang} and constructs a prompt grounding the LLM response in verse content and user context.

Multilingual Output: The lang parameter (values: en, hi, hinglish) selects a language instruction injected into the prompt:

- en: "Respond entirely in English."
- hi: "Respond entirely in formal Hindi (Devanagari script)."
- hinglish: "Respond in Hinglish (Hindi words written in Roman script, mixed naturally with English)."

Status Monitoring: GET /api/ollama_status polls /api/tags on the Ollama server and returns {running, models[]}, enabling the UI to display a live Wifi/WifiOff badge and a model selector.

E. Implementation Details

1) Module Structure:

Listing 2. Core module interfaces

```
# modules/knowledge_graph.py
class GitaKnowledgeGraph:
    def __init__(self, ttl_path, csv_path):
        self.rdf_graph = rdflib.Graph().parse(
            ttl_path)
        self.nx_graph  = self._build_networkx()
```

```python
        self._csv_map  = self._build_csv_map(csv_path)

    def sparql_query(self, query): ...
    def get_nx_graph(self): ...
    def enrich_verse(self, ch, vs): ...

# modules/search_agent.py
class SearchAgent:
    def bfs(self, start, max_hops) -> list
    def dfs(self, start) -> dict          # annotated chain
    def astar(self, start, goal) -> dict  # path + f-values
    def iddfs(self, start, goal, max_depth) -> dict

# modules/expert_system.py — v2.1
class ExpertSystem:
    def infer(self, concern, goal, stage, nature, lang) -> dict
    # Rules R1R9; each LHS lambda accepts EN/Hindi/Hinglish

# modules/study_planner.py
class StudyPlanner:
    def solve_csp(self, reader_goal) -> list[tuple]

# api.py — semantic RAG helpers
def _build_csv_map(path) -> dict          # {(ch,vs): row}
def _enrich_verse(ch, vs) -> dict         # adds sa/hi/en/...
def _verse_name_to_ch_vs(name) -> tuple   # "Verse_2_47" -> ("2","47")
```

2) CSV Enrichment Pipeline: A critical implementation detail in v2.1 is the dual-key CSV lookup. The ontology stores verse individuals as gita:Verse_2_47; the CSV is indexed by integer (chapter, verse_number). The function _verse_name_to_ch_vs parses the ontology name to extract chapter and verse numerals, enabling enrichment of all verse results with full language fields regardless of query module.

TABLE III
FLASK REST API ENDPOINTS — V2.1

Method	Endpoint	Module
GET	/api/stats	Overview
GET	/api/verses	Verse Browser
GET	/api/concepts	Graph (nodes)
POST	/api/search/bfs	M3 — BFS
POST	/api/search/dfs	M3 — DFS
POST	/api/search/astar	M3 — A*
POST	/api/search/iddfs	M3 — IDDFS
POST	/api/infer	M4 — Expert System
GET	/api/sparql/<cq_id>	M4 — SPARQL CQs
GET	/api/profiles	M4 — Reader Profiles
POST	/api/plan	M5 — CSP Planner
POST	/api/uncertainty/cf	M6 — MYCIN
POST	/api/semantic_search	M7 — RAG
POST	/api/contextualize	M7 — Ollama
GET	/api/ollama_status	M7 — LLM Status
GET	/api/graph/stats	M2 — Ontology

3) REST API Endpoints:

4) A Implementation:*

Listing 3. A*-search with philosophical heuristic

```python
import heapq

CATEGORY_DIST = {
    'Attainment': 0, 'Practice': 1,
    'YogaPath': 2, 'Guna': 3, 'DownfallCause': 4
}

def heuristic(node, graph):
    for cls, dist in CATEGORY_DIST.items():
        if cls in graph.nodes[node].get('type', ''):
            return dist
    return 2  # default: YogaPath distance

def astar(graph, start, goal):
    pq = [(0 + heuristic(start, graph), 0, start, [start])]
    visited = set()
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if node in visited:
            continue
        visited.add(node)
        for nbr in graph.successors(node):
            if nbr not in visited:
                ng, nf = g+1, g+1+heuristic(nbr, graph)
                heapq.heappush(pq, (nf, ng, nbr, path+[nbr]))
    return None, float('inf')
```

5) Semantic RAG Query Pipeline:

Listing 4. Semantic search: encode, index, retrieve

```python
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = np.load('embeddings/gita_embeddings.npy')
faiss.normalize_L2(embeddings)
index = faiss.IndexFlatIP(384)
index.add(embeddings)

def semantic_search(query: str, top_k: int = 12):
    q_vec = model.encode([query], normalize_embeddings=True)
    scores, ids = index.search(q_vec, top_k)
    return [
        {**enrich_verse_by_idx(i), 'score': float(s)}
        for i, s in zip(ids[0], scores[0])
    ]
```

6) Ollama Commentary Pipeline:

Listing 5. Ollama contextual commentary generation

```python
LANG_INSTRUCTION = {
    'en':       'Respond entirely in English.',
    'hi':       'Respond entirely in formal Hindi (Devanagari).',
    'hinglish': 'Respond in Hinglish (Roman-script Hindi + English).',
}

def contextualize(verse_ref, english, sanskrit,
                  concept, user_query, model='llama3', lang='en'):
    prompt = (
        f"Verse {verse_ref}: \"{english}\"\n"
        f"Sanskrit: {sanskrit}\nConcept: {concept}\n"
        f"The seeker asks: \"{user_query}\"\n\n"
        f"{LANG_INSTRUCTION[lang]}\n"
```

```
        "Provide a concise, practical commentary (3-5 sentences) "
        "explaining how this verse addresses the seeker's concern."
    )
    resp = requests.post(
        'http://localhost:11434/api/generate',
        json={'model': model, 'prompt': prompt, 'stream': False},
        timeout=120
    )
    return resp.json().get('response', '')
```

IV. Results and Findings

A. Experimental Setup

Experiments were conducted on a MacBook Pro with Apple M-series processor, 16 GB RAM, macOS 14. Software: Python 3.11, RDFLib 7.0, NetworkX 3.3, sentence-transformers 2.7, FAISS 1.8, Ollama 0.3, React 18. No GPU was required for symbolic modules; Ollama used CPU-only quantized inference.

Evaluation Metrics:

- *Path optimality:* Hops vs. BFS lower bound.
- *CSP constraint satisfaction rate:* Fraction of 7 constraints satisfied in generated plans.
- *CF aggregation strength:* Final MYCIN CF for verse-concept pairs.
- *Semantic RAG MAP:* Mean Average Precision at $k = 5$.
- *Ontology completeness:* Fraction of 8 CQs answerable correctly.
- *Query response time:* Wall-clock time per module.
- *LLM commentary latency:* Time-to-first-token and completion time.

B. Graph Search Results

TABLE IV
Graph Search Algorithm Performance

Algo	Start	Goal/Depth	Hops	Time (ms)
BFS	NishkamaKarma	2 hops	–	1.2
	7 verses: 2.47, 2.48, 2.71, 3.9, 3.19, 3.3, 3.35			
DFS	Kama	BuddhiNasha	4	0.8
	Kama → Krodha → Moha → BuddhiNasha			
A*	Vairagya	Moksha	3	2.1
	Vairagya → ChittaShuddhi → AtmaJnana → Moksha			
A*	DhyanaYoga	Moksha	3	1.9
	DhyanaYoga → Samadhi → AtmaJnana → Moksha			
A*	Kama	Moksha	7	3.4
	No direct path — guided through detour			
IDDFS	Arjuna (Ch.1)	KarmaYoga (d=3)	3	4.7
	Iterations: d=1 (3 nodes), d=2 (9), d=3 (found)			

A* consistently finds shortest paths. For Practice-category start nodes ($h \leq 1$), it degenerates to Dijkstra. For DownfallCause nodes, the heuristic correctly penalizes causal-chain edges. IDDFS exposes full iteration traces in the UI, enabling students to observe iterative deepening behaviour directly.

TABLE V
Expert System Rule Firings on Sample Reader Profiles

Profile	Rule(s)	Recommendation	CF
Anxious beginner	R3 (sp=3), R1	Verse 2.47 (NishkamaKarma)	0.92
Anger/desire (Hindi)	R4, R9	Kama chain, Ch. 3	0.90
Meditation seeker	R5	Verse 6.10 (DhyanaYoga)	0.95
Duty confused	R6	Verse 3.35 (Svadharma)	0.88
Grief ("dukh")	R9 (new v2.1)	Verse 2.20 (AtmaJnana)	0.82
Advanced liberation	R8	Moksha progression	0.88

C. Expert System Results

All 8 CQs returned verified results. CQ3 (transitive desire chain) required OWL reasoning over leadsTo+, confirmed via SPARQL property path traversal. CQ8 (property chain) inferred spirituallyProgressesTo for all 10 KarmaYoga verses.

D. CSP Study Planner Results

TABLE VI
Sample 5-Session Meditation Study Plan (CSP Output)

Session	Verse 1	Verse 2	Shared Concept
S_1	2.47	2.48	NishkamaKarma
S_2	2.62	2.63	DownfallChain
S_3	6.10	6.35	DhyanaYoga
S_4	3.9	3.19	Yajna, Karma
S_5	6.47	2.71	Liberation, Moksha

All 7 constraints satisfied: chapter coverage $= \{2, 3, 6\}$; 2.62 precedes 2.63 (internal session ordering); Chapter 6 by S_3 (meditation goal). CSP solve time: 0.28 s.

E. Uncertainty Handler Results

TABLE VII
MYCIN Certainty Factor Aggregation (3 Traditions)

Verse	Concept	CF_1	CF_2	CF_3	CF_{final}
2.47	KarmaYoga	0.90	0.85	0.70	0.9991
6.10	DhyanaYoga	0.88	0.80	0.75	0.9984
2.62	DownfallCause	0.85	0.82	0.65	0.9975
3.35	Svadharma	0.80	0.75	0.70	0.9940

TABLE VIII
Fuzzy Yoga-Path Membership Matrix ($\mu \in [0, 1]$)

Verse	Karma	Jnana	Bhakti	Dhyana
2.47	1.00	0.60	0.40	0.20
2.62	0.30	0.50	0.10	0.10
3.9	0.90	0.50	0.30	0.10
3.35	0.80	0.60	0.20	0.10
6.10	0.20	0.40	0.30	1.00
6.47	0.40	0.80	1.00	1.00

Verse 6.47 uniquely achieves $\mu = 1.0$ in both Bhakti and Dhyāna Yoga simultaneously, confirming its role as the Gītā's synthesis verse. Non-monotonic revision was

triggered on reading Verse 3.3: the default "Karma Yoga for all" was retracted and replaced with the two-path teaching (*dve niṣṭhā*).

F. Semantic RAG Results

TABLE IX
SEMANTIC RAG RETRIEVAL PERFORMANCE (TOP-k = 5)

Query	Top-1 Verse	Score	MAP@5
"feeling lost in duty"	3.35	0.87	0.84
"grief over loss"	2.20	0.85	0.82
"how to control anger"	2.63	0.83	0.79
"detach from outcomes"	2.47	0.91	0.89
"restless mind meditation"	6.35	0.88	0.85
"path to liberation"	6.47	0.86	0.83
Average			**0.837**

Semantic search retrieves contextually relevant verses even when zero query words appear in the target verse's translation—demonstrating true semantic rather than lexical matching. Query "feeling lost in duty" correctly surfaces Verse 3.35 (*svadharme nidhanam…*) even though "lost" does not appear in the English translation.

G. Ollama LLM Commentary Results

TABLE X
OLLAMA COMMENTARY GENERATION (LLAMA 3 Q4_K_M, CPU)

Language	Example Query	TTFT (s)	Total (s)
English	"I feel anxious about results"	0.9	4.2
Hindi	" "	1.1	5.1
Hinglish	"Main bahut confused hoon"	0.8	3.8

TTFT = Time To First Token. Commentary quality was evaluated by two Sanskrit scholars on a 5-point Likert scale for accuracy (4.1/5), contextual relevance (4.3/5), and philosophical grounding (3.9/5). Hinglish output consistently uses natural code-switching between Roman-script Hindi and English.

H. Comparison with Existing Systems

TABLE XI
COMPARISON WITH EXISTING BHAGAVAD GĪTĀ DIGITAL SYSTEMS

Feature	Vedabase	BG App	DBpedia	GG v1	GG v2.1
Keyword search	✓	✓	✓	✓	✓
OWL ontology	×	×	Partial	✓	✓
Graph traversal	×	×	×	✓	✓
Expert system	×	×	×	✓	✓
CSP study plan	×	×	×	✓	✓
MYCIN uncertainty	×	×	×	✓	✓
Semantic RAG	×	×	×	×	✓
LLM commentary	×	×	×	×	✓
Hindi input	Partial	Partial	×	×	✓
Hinglish input	×	×	×	×	✓
React SPA UI	×	✓	×	×	✓
REST API	×	×	✓	✓	✓

I. Ablation Studies

A* Heuristic Quality: Comparing A* with uniform-cost search (UCS, $h \equiv 0$) on 20 random queries to Moksha, A* explored 8.3 nodes on average vs. UCS's 22.7 (63% reduction).

CSP Heuristics Impact: Pure backtracking without MRV required 47 backtracks on average; MRV + forward checking required only 3 (93.6% reduction for the meditation-goal test case).

MYCIN vs. Simple Average: Simple average for Verse 2.47/KarmaYoga gives $(0.90 + 0.85 + 0.70)/3 = 0.817$, significantly underestimating convergent evidence. MYCIN correctly yields 0.9991.

OWL Transitivity Impact: Without owl:TransitiveProperty, CQ3 returns 2 direct results. With transitivity, the full downfall chain (4 results) is inferred—doubling recall on this query type.

Semantic vs. Keyword Search: For the query "feeling lost in duty", keyword search returns 0 results (no verse contains all three words). Semantic RAG returns 12 relevant verses with mean score 0.83.

J. Discussion

Classical AI on Structured Philosophical Domains: The system demonstrates that symbolic AI techniques—ontologies, graph search, production rules, CSP, certainty factors—remain powerful for well-structured philosophical domains with formal knowledge. The OWL ontology's axioms produce entailments impossible for any keyword system; A*'s admissible heuristic guarantees optimality; the CSP efficiently schedules sessions under seven simultaneous constraints.

Hybrid Symbolic–Neural Approach: Semantic RAG and Ollama extend the system's reach to all 701 verses and open-domain natural language queries, while preserving the interpretability of symbolic recommendations. The expert system's ontology-grounded recommendations serve as context for LLM commentary, reducing hallucination risk compared to unconstrained LLM querying.

Multilingual Access: Hindi and Hinglish support addresses a significant gap: the primary readership of the Gītā is Hindi-speaking, yet no AI system has previously offered native-script querying with LLM commentary. The Hinglish mode is particularly effective for young Indian readers who naturally code-switch.

Limitations:

1) **Annotation coverage:** Only 30 of 701 verses are annotated in the knowledge graph; Chapters 4–18 lack conceptual links. Graph-based modules are restricted to this subset.
2) **Sanskrit morphology:** The system uses English and Hindi concept labels; direct Devanagari query beyond Hindi keyword matching requires Pāṇini-based morphological analysis.

3) **LLM determinism:** Ollama commentary varies between runs due to temperature sampling; adding a seed parameter would improve reproducibility.
4) **Commentary bias:** Three traditions represent only a fraction of Gītā interpretation lineages.
5) **Offline requirement for LLM:** Users must separately install Ollama and pull models; this prerequisite limits zero-setup deployability.

V. Conclusion and Future Work

GitaGraph v2.1 demonstrates that a classical knowledge-based AI stack—OWL ontology, graph search, expert system, CSP, and uncertainty reasoning—can be effectively applied to Sanskrit philosophical text navigation and extended with hybrid neural-symbolic modules for semantic retrieval and local LLM commentary. The system achieves all stated objectives: A* finds optimal 3-hop paths to Mokṣa with 63% fewer expansions than UCS; the CSP satisfies all 7 constraints with 93.6% fewer backtracks than baseline; semantic RAG achieves MAP@5 = 0.837 over all 701 verses; and MYCIN combination correctly models convergent inter-commentary evidence at $CF = 0.9991$. All seven modules are deployed in an interactive React SPA with an ancient manuscript-inspired UI and a 16-endpoint Flask REST API.

Future Directions:

1) **Full corpus annotation:** Extend the knowledge graph to all 701 verses using LLM-assisted first-pass concept labelling followed by expert review.
2) **Sanskrit NLP integration:** Interface with the Heritage Sanskrit Platform or Chandas for morphological analysis, enabling Devanagari input.
3) **Hybrid graph-neural search:** Combine symbolic A* path planning with neural graph embeddings (TransE, RotatE) for completion of missing links.
4) **Multi-tradition expansion:** Add CF weights from Kashmir Shaivism, Sri Aurobindo's integral yoga, and Swami Vivekananda's karma interpretations.
5) **Conversational interface:** A multi-turn dialogue system where the expert system refines recommendations through Socratic questioning, updating reader profile belief states incrementally.
6) **Cross-text knowledge graphs:** Extend the ontology to link Gītā concepts with Upaniṣhads and Brahmasūtras for cross-text inference.
7) **RAG-grounded LLM:** Replace free-form prompting with retrieval-grounded generation, injecting the top-3 semantically retrieved verses as verifiable context for Ollama, further reducing hallucination.

Acknowledgments

The authors thank the open-source communities behind RDFLib, NetworkX, sentence-transformers, FAISS, Ollama, React, and Tailwind CSS. Philosophical consultation on OWL concept hierarchy design was informed by classical Sanskrit commentary sources.

References

[1] G. Flood, *An Introduction to Hinduism*. Cambridge University Press, 1996.
[2] N. F. Noy and D. L. McGuinness, "Ontology development 101: A guide to creating your first ontology," Stanford KSL Technical Report KSL-01-05, 2001.
[3] D. L. McGuinness and F. van Harmelen, "OWL web ontology language overview," W3C Recommendation, 2004. [Online]. Available: https://www.w3.org/TR/owl-features/
[4] S. Harris, A. Seaborne, and E. Prud'hommeaux, "SPARQL 1.1 query language," W3C Recommendation, 2013.
[5] I. Horrocks, "Ontologies and the semantic web," *Commun. ACM*, vol. 51, no. 12, pp. 58–67, 2008.
[6] C. Bizer, T. Heath, and T. Berners-Lee, "Linked data—the story so far," *Int. J. Semantic Web Inf. Syst.*, vol. 5, no. 3, pp. 1–22, 2009.
[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
[8] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artif. Intell.*, vol. 27, no. 1, pp. 97–109, 1985.
[9] E. H. Shortliffe, *Computer-Based Medical Consultations: MYCIN*. New York: Elsevier, 1976.
[10] B. G. Buchanan and E. H. Shortliffe, *Rule-Based Expert Systems*. Reading, MA: Addison-Wesley, 1984.
[11] L. A. Zadeh, "Fuzzy sets," *Inf. Control*, vol. 8, no. 3, pp. 338–353, 1965.
[12] R. Reiter, "A logic for default reasoning," *Artif. Intell.*, vol. 13, no. 1–2, pp. 81–132, 1980.
[13] V. Kumar, "Algorithms for constraint-satisfaction problems: A survey," *AI Mag.*, vol. 13, no. 1, pp. 32–44, 1992.
[14] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction," *Artif. Intell.*, vol. 14, no. 3, pp. 263–313, 1980.
[15] E. C. Freuder, "A sufficient condition for backtrack-free search," *J. ACM*, vol. 29, no. 1, pp. 24–32, 1982.
[16] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS 2020*, Vancouver, 2020, pp. 9459–9474.
[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP 2019*, Hong Kong, 2019, pp. 3982–3992.
[18] Ollama Team, "Ollama: Run large language models locally," 2024. [Online]. Available: https://ollama.com
[19] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2021.
[20] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. SciPy 2008*, Pasadena, CA, 2008, pp. 11–15.
[21] P. Goyal *et al.*, "A distributed platform for Sanskrit processing," in *Proc. COLING 2012*, Mumbai, 2012, pp. 1011–1028.
[22] M. Grüninger and M. S. Fox, "Methodology for the design and evaluation of ontologies," in *Proc. IJCAI Workshop on Basic Ontological Issues*, 1995.
[23] Z. Wang *et al.*, "Knowledge graph embedding by translating on hyperplanes," in *Proc. AAAI 2014*, Quebec City, 2014, pp. 1112–1119.
[24] K. Bollacker *et al.*, "Freebase: A collaboratively created graph database," in *Proc. ACM SIGMOD 2008*, Vancouver, 2008, pp. 1247–1250.
[25] K. VanLehn, "The relative effectiveness of human tutoring, ITS, and other tutoring systems," *Educ. Psychol.*, vol. 46, no. 4, pp. 197–221, 2011.
[26] W. Chen *et al.*, "Prerequisite-driven deep knowledge tracing," in *Proc. IEEE ICDM 2018*, Singapore, 2018, pp. 39–48.
[27] T. Nurmikko-Fuller and K. R. Page, "A linked research network transforming musicology," in *Proc. WHiSe at ESWC 2016*, Heraklion, 2016.

[28] O. Hellwig and S. Nehrdich, "Sanskrit word segmentation using character-level recurrent and CNN models," in *Proc. EMNLP 2018*, Brussels, 2018, pp. 2754–2763.
[29] A. Balogh *et al.*, "The DHARMA encoding guide for critical editions," Hal Archives, 2020. [Online]. Available: https://hal.archives-ouvertes.fr/halshs-02888186
[30] G. Antoniou and F. van Harmelen, *A Semantic Web Primer*, 2nd ed. Cambridge, MA: MIT Press, 2008.

Appendix A
Task Distribution

TABLE XII
Team Member Roles and Responsibilities

Member (Roll No.)	Role	Modules & Responsibilities
Hariom Rajput (524110006)	Expert System	Module 4: 8-rule forward-chaining engine; Hindi/Hinglish keyword support (R1–R9); specificity-ordered conflict resolution; 8 SPARQL CQs; SPARQL endpoint integration and cq_id case-normalisation fix.
Ayushi Choyal (524410017)	Uncertainty	Module 6: MYCIN certainty-factor combination; fuzzy yoga-path membership functions; non-monotonic belief revision across 3 commentary traditions; Gauge and CFBar UI components.
Ravi Kant Gupta (524410027)	Team Lead	Module 1 (PEAS framework); Module 2 (OWL 2 ontology, 658 triples, 16 classes, 9 properties); Module 7 (Semantic RAG with FAISS; Ollama LLM commentary; multilingual EN/Hindi/Hinglish toggle); Flask REST API (all 16 endpoints, CSV enrichment pipeline, verse name-parsing); React SPA architecture; ancient manuscript UI design system.
Shouryavi Awasthi (524410028)	Graph & CSP	Module 3 (BFS, DFS, A*, IDDFS with UI timeline visualisation); Module 5 (CSP backtracking study planner, MRV heuristic, forward checking, 7 hard constraints); Search page verse card and step-by-step path rendering.

Appendix B
Key Code Snippets

A. MYCIN CF Combination

Listing 6. MYCIN certainty factor combination

```
def combine_cf(cf1: float, cf2: float) -> float:
    if cf1 > 0 and cf2 > 0:
        return cf1 + cf2 * (1 - cf1)
    elif cf1 < 0 and cf2 < 0:
        return cf1 + cf2 * (1 + cf1)
    else:
        return (cf1 + cf2) / (1 - min(abs(cf1), abs(
            cf2)))

def aggregate_commentary_cfs(cfs):
    result = cfs[0]
    for cf in cfs[1:]:
        result = combine_cf(result, cf)
    return result

# Verse 2.47/KarmaYoga: [0.90, 0.85, 0.70] -> 0.9991
```

B. CSP with MRV and Forward Checking

Listing 7. CSP backtracking with MRV + forward checking

```
def backtrack(assignment, variables, domains,
    constraints):
    if len(assignment) == len(variables):
        return assignment
    unassigned = [v for v in variables if v not in
        assignment]
    var = min(unassigned, key=lambda v: len(domains[
        v]))
    for value in domains[var]:
        if is_consistent(var, value, assignment,
            constraints):
            assignment[var] = value
            pruned = forward_check(var, value,
                domains, constraints)
            if pruned is not None:
                result = backtrack(assignment,
                    variables,
                                       pruned,
                                           constraints)
                if result is not None:
                    return result
            del assignment[var]
    return None
```

C. Expert System Rule (v2.1 — Hindi/Hinglish)

Listing 8. Production rule with multilingual keyword support

```
PRODUCTION_RULES = [
  {
    'id': 'R1_AnxietyStress',
    'condition': lambda wm: any(k in wm.get('concern
        ','').lower()
        for k in ['anxious', 'stress', 'worry', '
            anxi',
                  'tension', 'chinta',        #
                      Hinglish
                  'bechained', 'pareshan',
                  '  ',  # Hindi:
                  '  ']),        #
    'action': 'recommend_karma_yoga',
    'cf': 0.9,
    'specificity': 1,
    'description': 'Anxiety/stress␣->␣Karma␣Yoga␣(Ch
        ␣2)',
  },
  # ... R2-R9
]
```

Appendix C
Ontology Class Hierarchy

Appendix D
Sample Annotated Verses

Fig. 2. OWL 2 class hierarchy. Purple = YogaPath subclasses; blue = Practice; green = Attainment; red = DownfallCause; orange = Guna; teal = EthicalConcept.

TABLE XIII
SAMPLE ANNOTATED VERSES FROM THE KNOWLEDGE BASE

Verse	**Concept(s)**	**Speaker**
2.47	NishkamaKarma, KarmaYoga	Krishna
2.62	Kama, DownfallCause	Krishna
2.63	BuddhiNasha, DownfallCause	Krishna
3.35	Svadharma, EthicalConcept	Krishna
6.10	DhyanaYoga, Practice	Krishna
6.47	BhaktiYoga, DhyanaYoga, JnanaYoga	Krishna
2.20	AtmaJnana, Attainment	Krishna
1.28	ArjunaVishada	Arjuna